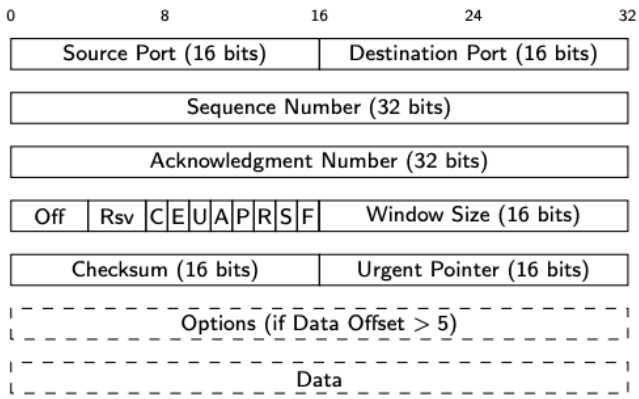
Lecture 19 → TCP Part 1

Acronym	Meaning
ACK	Acknowledgment
EWMA	Exponentially Weighted Moving Avg
FIN	Finish (close connection)
ISN	Initial Sequence Number
MSL	Maximum Segment Lifetime
MSS	Maximum Segment Size

Acronym	Meaning
RTO	Retransmission Timeout
RTT	Round-Trip Time
RST	Reset
rwnd	Receive Window
SACK	Selective ACK
SRTT	Smoothed RTT (filtered estimate)

TCP Segment Header → 20 bytes minimum



Header Fields: Sequence, ACK, Flags, Offset

- Sequence Number (32 bits)
  - Byte offset of the first byte of data in this segment
  - SYN and FIN each consume one sequence number
- Acknowledgment Number (32 bits)
  - Next byte the sender of this segment expects to receive (cumulative ACK)
  - ACK = N means “I have all bytes through N − 1.”
  - Valid only when ACK flag = 1
- Data Offset (4 bits)
  - Header length in 32-bit words (like IHL in IPv4)
  - Min = 5 (20 bytes); max = 15 (60 bytes)

Key flags

Flag	Meaning
SYN	Connection initiation; carries ISN
ACK	ACK number field is valid
FIN	Sender has no more data
RST	Abort connection immediately
PSH	Deliver to app without buffering
URG	Urgent pointer is valid
CWR/ECE	Congestion signals (L20)

Header Fields: Window, Checksum, Options

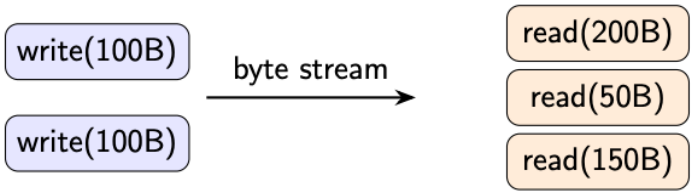
- Window Size (16 bits)
  - rwnd Free buffer space at the receiver

Key TCP options

Option	Purpose
MSS	Max segment size; exchanged at SYN
Window Scale	Multiplies window by 2^n; up to 1 GB
SACK	SR-style: ACK specific byte ranges*
Timestamps	Precise RTT; removes Karn ambiguity

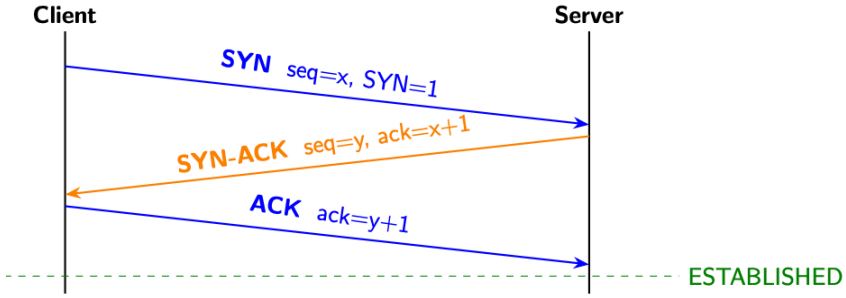
        - Sender must keep bytes-in-flight ≤ rwnd
        - Limits to 64 KB without Window Scale option
    - Checksum (16 bits)
        - Covers pseudo-header (src IP, dst IP, proto=6, TCP length) + header + data
        - Mandatory in both IPv4 and IPv6 (unlike UDP's optional checksum)

The Byte Stream Model → TCP delivers a continuous byte stream → there are no message boundaries at the TCP level



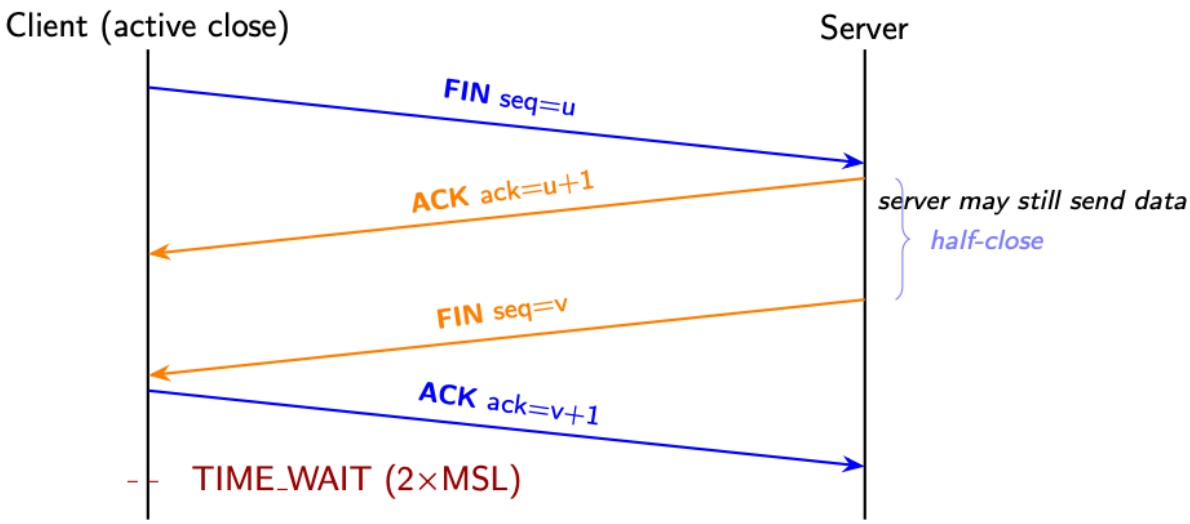
Sequence numbers count bytes andnot segments
- ISN = 1000; first segment carries 500 bytes of data
- Seq = 1000; payload = bytes 1000–1499
- Next segment: seq = 1500
- ACK from receiver: ack = 1500 ("send me byte 1500 next")

3 Way handshake → Both sides must agree on ISNs and negotiate options before data flows → minimum messages for reliable mutual agreement: three



ISN Randomization
Why not start every connection at seq = 0?
Problem 1: Stale segments
Problem 2: Security

Connection Teardown: FIN?ACK Exchange → Each direction closes independently ("half-close")



TIME WAIT and RST

<u>TIME WAIT</u> → Duration: 2 × MSL (typically 2–4 minutes)

Why it must exist:

1. Final ACK may be lost Server retransmits FIN. Client must still be alive to re-ACK.
2. Prevent 5-tuple reuse confusion Stale segments from the old connection must expire before a new connection reuses the same addresses and ports

<u>RST</u>: Abrupt close

Sent when:

1. Connection request arrives at a port with no listener ("Connection refused")
2. One side crashed and restarted → half-open detection
3. Firewall terminating a connection

RST is not acknowledged

Reliable Delivery in TCP

<u>Cumulative ACKs (GBN-style)</u> → ACK = N → "I have all bytes through N − 1."

- One ACK can acknowledge many bytes
- Delayed ACK: receiver waits up to 500 ms (or 2 segments) before ACKing → reduces ACK traffic
- SACK option adds SR-style selective acknowledgment for specific byte ranges

<u>Retransmission timer (RTO)</u> → One timer per connection, tracking oldest unACKed byte

- RTO fires → retransmit; double the RTO (backoff)
- RTO must track changing network RTT
- Too short: spurious retransmits, wastes bandwidth
- Too long: slow recovery from real loss

RTT Estiamtion: EWMA as a Low Pass FIlter

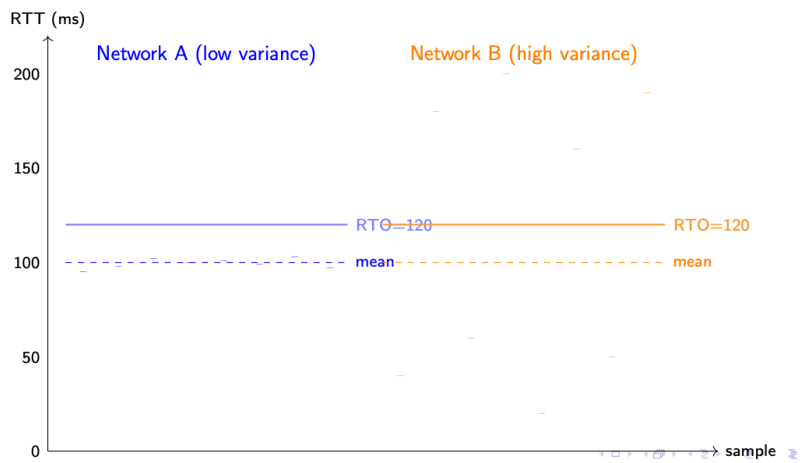
TCP tracks RTT with an Exponentially Weighted Moving Average:

$$\text{SRTT} \leftarrow (1-\alpha)\cdot\text{SRTT} + \alpha\cdot R \qquad \alpha = \frac{1}{8}$$

$$\text{SRTT}_n = \alpha\sum_{k=0}^{n}(1-\alpha)^k \cdot R_{n-k}$$

Expanding the recursion:
Time constant ≈ 1/α = 8 sample

Why the Mean is Not Enough: Variance Matters → Two networks, both with mean RTT = 100 ms



Jacobson's Fix: RTTVAR and the RTO Formula

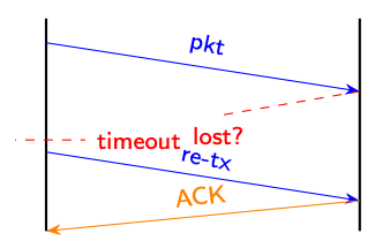
track the mean absolute deviation of the RTT: $\text{RTTVAR} \leftarrow (1-\beta)\cdot\text{RTTVAR} + \beta\cdot|\text{SRTT} - R| \qquad \beta = \frac{1}{4}$

RTTVAR is another EWMA → this one tracking variability (noise power): $\boxed{\text{RTO} = \text{SRTT} + 4\cdot\text{RTTVAR}}$
Engineering interpretation: RTO = mean + 4×variability

The two-filter system:

SRTT	tracks the signal mean
RTTVAR	tracks the noise power
RTO	mean + 4×noise

Karns Algorithm and Exponential Backoff

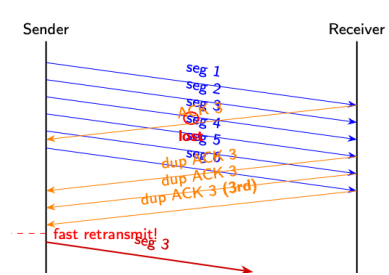


Which tx? **Unknown.**

Karn's algorithm → When a retransmit occurs, an ACK arrives, but which transmission did it acknowledge?
Rule: Do not update SRTT/RTTVAR from any ACK following a retransmit
Exponential backoff Each consecutive timeout:

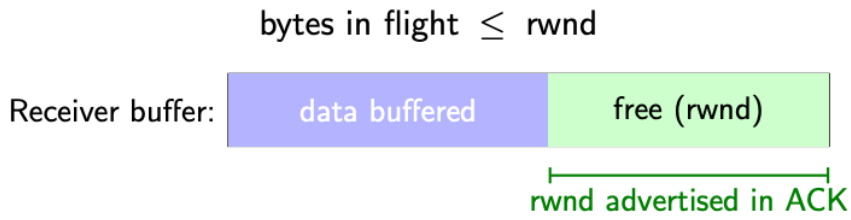
RTO ← 2 × RTO

Fast Retransmit: Dont Wait for the Timer



When a receiver gets an out-of-order segment, it immediately sends an ACK for the last in-order byte → a duplicate ACK
Rule: 3 duplicate ACKs → retransmit immediately, without waiting for RTO

Flow Control: The receive window
Problem: sender and receiver run at different speeds. If the sender transmits faster than the application reads, the receive buffer overflows
Solution: every TCP segment carries rwnd → the receiver's available buffer space



Flow Control: Worked Example

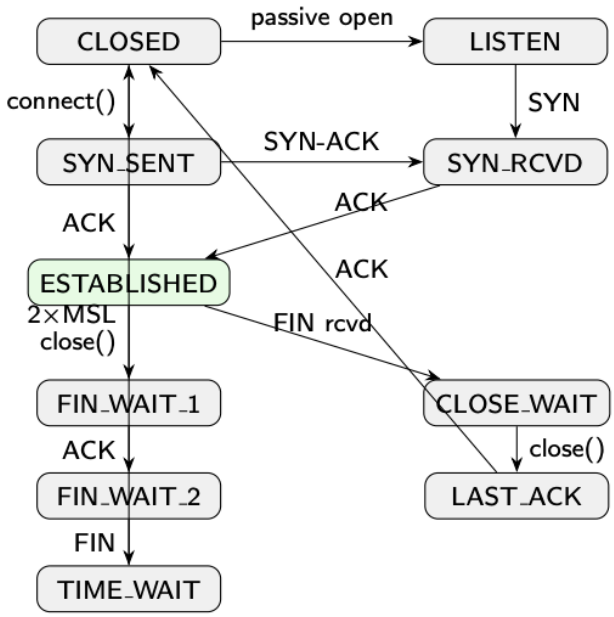
After 3 segments arrive (1460 B each):
Data buffered: $3 \times 1460 = 4380$ B
Free space: $16384 - 4380 = 12004$ B
⇒ ACK carries **rwnd = 12004**

Application reads 4380 B:
Data buffered: 0 B
Free space: 16384 B
⇒ ACK carries **rwnd = 16384**

Buffer fills completely:
⇒ ACK carries **rwnd = 0**
Sender pauses; sends 1-byte probes until receiver reopens window

Nagle Algorithm
Problem: interactive applications (SSH, Telnet) send many tiny writes. Each keystroke becomes a 41-byte segment (1 byte data + 40 bytes headers) → wasteful
Nagle's algorithm: if there is data already in flight (unACKed), buffer small writes until an ACK arrives, then send everything accumulated as one segment
Benefit: Reduces small-packet flooding on slow or congested links
Cost: Adds latency → the buffered data waits for an ACK

TCP State Machine


CLOSED
passive open
LISTEN
connect()
SYN
SYN_SENT
SYN-ACK
SYN_RCVD
ACK
ACK
ESTABLISHED
ACK
2×MSL
close()
FIN rcvd
FIN_WAIT_1
CLOSE_WAIT
ACK
close()
FIN_WAIT_2
LAST_ACK
FIN
TIME_WAIT